

Transmisje pay per view

Autorzy:

Piotr Raczkowiak 140587

Krystian Frąckowiak 140476

Miłosz Gałęcki 138304

Opis najważniejszych założeń i wymagań

Przedmiotem projektu jest system obsługujący transmisję typu Pay Per View (PPV). Ze względu na krytyczny charakter aplikacji, której niedostępność może prowadzić do utraty przychodów oraz problemów z obsługą klientów, głównym celem projektu jest zapewnienie wysokiej dostępności, wydajności oraz bezpieczeństwa danych.

Podstawą rozwiązania jest system bazodanowy PostgreSQL działający w architekturze rozproszonej. Dane aplikacji przechowywane są na serwerze głównym (Primary/Master), który odpowiada za wszystkie operacje zapisu do bazy danych.

W celu zwiększenia wydajności systemu zastosowano dwa serwery replik PostgreSQL działające w trybie tylko do odczytu (Standby Read1 oraz Standby Read2). Dane są synchronizowane z serwera głównego za pomocą mechanizmu fizycznej replikacji strumieniowej (Streaming Replication). Dzięki temu możliwe jest odciążenie serwera głównego poprzez kierowanie zapytań odczytujących do replik.

Rozdzielenie ruchu pomiędzy operacje zapisu i odczytu realizowane jest przez dwa niezależne serwery HAProxy. Pierwszy serwer HAProxy obsługuje wyłącznie operacje zapisu (INSERT, UPDATE, DELETE) i przekazuje je do serwera głównego PostgreSQL. Drugi serwer HAProxy obsługuje zapytania odczytujące (SELECT) i rozkłada ruch pomiędzy dwie repliki odczytowe. Takie rozwiązanie zwiększa wydajność systemu oraz umożliwia lepsze wykorzystanie dostępnych zasobów.

W celu zapewnienia wysokiej dostępności oraz ochrony przed awarią całej lokalizacji zastosowano dodatkowy serwer zapasowy (Disaster Recovery), znajdujący się w odrębnej lokalizacji logicznej. Serwer ten utrzymuje aktualną kopię danych dzięki wykorzystaniu mechanizmu replikacji PostgreSQL i może zostać wykorzystany do odtworzenia działania systemu w przypadku awarii serwera głównego. Regularnie wykonywane kopie bezpieczeństwa umożliwiają odtworzenie bazy danych po awarii sprzętowej, błędzie administracyjnym lub przypadkowym usunięciu danych.

W projekcie wykorzystano również mechanizmy bezpieczeństwa dostępne w PostgreSQL, takie jak podział użytkowników na role o różnych poziomach uprawnień, ograniczanie dostępu do bazy danych wyłącznie z określonych adresów IP oraz bezpieczne uwierzytelnianie użytkowników przy użyciu algorytmu SCRAM-SHA-256.

Diagram

